Ordena tu vida con Claude: 4 sistemas en un fin de semana

4 prompts para que Claude Code construya y publique 4 apps web personales. Cada una es independiente, arma solo la que te sirva.

Herramientas

- **Claude Code:** construye la app y la publica.
- **GitHub:** guarda el código.
- **Vercel:** pone la app en internet y corre las tareas automáticas.
- **Supabase:** base de datos, login y archivos.
- **API de Anthropic:** la IA dentro de la app.
- **Resend:** manda los correos automáticos.
- **Google Cloud:** solo para conectar Calendar y Gmail en el Panel de vida.

Cómo usarlos

1. Abre Claude Code en una carpeta vacía (una por app).
2. Llena los campos [entre corchetes] del prompt con tus datos.
3. Pega esta frase al inicio del prompt para que Claude te guíe paso a paso:

```
Soy principiante. Guíame paso a paso y espera a que te confirme que terminé cada acción antes de seguir.
```

4. Aprueba el plan que te muestra Claude y deja que construya y publique la app. Si sale un error, pégaselo y pídele que te explique cómo arreglarlo.

Detalle a considerar

El Panel de vida es el más técnico porque requiere configurar Google Cloud. Si es tu primera vez, empieza por cualquiera de los otros tres.

Prompt 1: Panel de vida

Junta tu calendario, correo y tareas en una página y te manda un resumen cada mañana.

panel_de_vida.xml

```
<objetivo>
Construye y despliega en Vercel una aplicación web personal llamada "Panel de Vida": una sola página que junta mi calendario, mi correo, mis tareas y mis herramientas conectadas, y que me envía un resumen por correo cada mañana antes de que empiece mi día.
```

```
</objetivo>

<stack>
- Next.js (App Router) con TypeScript
- Tailwind CSS y shadcn/ui
- Supabase: Postgres, Auth y Row Level Security
- Google APIs: Calendar API y Gmail API (solo lectura)
- Anthropic API para clasificar correos y redactar el resumen (modelo en la
variable ANTHROPIC_MODEL, valor por defecto claude-sonnet-5-5)
- Resend para enviar el resumen por correo
- Vercel Cron Jobs para la ejecución diaria
</stack>

<usuario>
- Uso personal, un solo usuario: [TU CORREO]
- Zona horaria: [EJ. America/Mexico_City]
- Hora del resumen matutino: [EJ. 6:45 am]
- Remitentes prioritarios: [EJ. jefe@empresa.com, socio@empresa.com]
</usuario>

<autenticacion_e_integraciones>
1. Login con Supabase Auth usando Google. Solo permite el acceso a mi correo;
cualquier otro correo recibe una pantalla de "acceso denegado".
2. Flujo OAuth de Google con los scopes calendar.readonly y gmail.readonly. Guarda
el refresh token cifrado en una tabla de Supabase con RLS.
3. Implementa la renovación automática del access token antes de cada llamada.
4. Deja la app de Google Cloud en modo "Testing" con mi correo como usuario de
prueba y explícame en el README por qué (los scopes de Gmail requieren verificación
de Google para uso público).
</autenticacion_e_integraciones>

<modelo_de_datos>
Crea las migraciones de Supabase para:
- tasks: id, user_id, titulo, descripcion, fecha_limite, prioridad
(alta/media/baja), estado (pendiente/en_progreso/hecha), proyecto, created_at
- email_classifications: id, user_id, gmail_thread_id, categoria
(responder_hoy/esperando_respuesta/informativo), resumen_una_linea,
fecha_limite_detectada, updated_at
- daily_briefs: id, user_id, fecha, contenido_markdown, enviado (boolean),
created_at
- integrations: id, user_id, proveedor, refresh_token_cifrado,
```

```
ultima_sincronizacion
Activa RLS en todas las tablas para que cada fila solo sea visible para su user_id.
</modelo_de_datos>

<pagina_principal>
Ruta "/" con estas secciones en este orden:

<seccion nombre="hoy_en_una_linea">
Una frase generada a partir de los datos: número de reuniones, horas libres y la
tarea más importante.
</seccion>

<seccion nombre="agenda">
Línea de tiempo vertical del día con los eventos de Google Calendar: hora, título,
duración, enlace de videollamada. Resalta conflictos de horario y eventos con menos
de 15 minutos de separación. Muestra los bloques libres de más de 45 minutos como
"tiempo disponible". Incluye un selector para ver los próximos 7 días.
</seccion>

<seccion nombre="correos">
Lee los hilos no leídos o sin respuesta de las últimas 48 horas, más los de
remitentes prioritarios. Envía asunto, remitente y fragmento a la API de Anthropic
con salida JSON estructurada para clasificarlos:
- responder_hoy: pregunta directa, solicitud con fecha o remitente prioritario
- esperando_respuesta: yo envié el último mensaje hace más de 3 días
- informativo: boletines y notificaciones (solo mostrar el conteo)
Guarda el resultado en email_classifications para no reclasificar hilos ya
procesados. Cada tarjeta enlaza al hilo en Gmail.
</seccion>

<seccion nombre="tareas">
CRUD completo de tareas en tres grupos: vencidas, hoy y próximos 3 días. Permite
crear una tarea en una sola línea con fecha en lenguaje natural (ej. "pagar luz el
viernes"), interpretada por la API de Anthropic.
</seccion>

<seccion nombre="alertas_cruzadas">
Lógica que cruza las fuentes y detecta:
- Correos con una fecha o reunión que no existe en el calendario
- Reuniones de hoy sin enlace ni descripción
- Tareas que vencen hoy sin tiempo libre suficiente en la agenda
```

```
- Días de la semana con más de 6 horas de reuniones
</seccion>

<seccion nombre="estado_de_conexiones">
Lista de integraciones con su última sincronización y un botón para reconectar.
</seccion>
</pagina_principal>

<resumen_matutino>
1. Crea la ruta /api/cron/daily-brief protegida con la variable CRON_SECRET.
2. Configura vercel.json con un cron que se ejecute a la hora del resumen
convertida a UTC.
3. La ruta obtiene agenda, correos clasificados, tareas y alertas, y le pide a la
API de Anthropic un resumen con esta estructura: 3 prioridades del día con el
motivo, agenda resumida, máximo 5 correos de responder_hoy, alertas, y una
sugerencia para reorganizar el día si hay sobrecarga. Debe poder leerse en menos de
90 segundos.
4. Guarda el resumen en daily_briefs y envíalo a mi correo con Resend usando una
plantilla HTML sencilla.
5. Agrega un botón "Enviar resumen ahora" en la página para probarlo manualmente.
</resumen_matutino>

<diseno>
- Paleta neutra: negro, grises y blanco. Sin colores de acento llamativos.
- Tipografía legible, tarjetas con bordes sutiles, modo claro y oscuro.
- Responsive: debe funcionar bien en el celular.
</diseno>

<seguridad>
- La app nunca envía, borra ni modifica correos ni eventos; solo lectura.
- Todas las llaves en variables de entorno, nunca en el código.
- Valida CRON_SECRET en cada ejecución del cron.
</seguridad>

<flujo_de_trabajo>
1. Antes de escribir código, muéstrame el plan: estructura de carpetas,
migraciones, rutas y la lista de variables de entorno. Espera mi aprobación.
2. Construye por fases: (a) proyecto base y login, (b) Google OAuth y agenda, (c)
correos y clasificación, (d) tareas y alertas, (e) cron y resumen por correo.
3. Al terminar cada fase, corre el build y los tipos, corrige errores y dime qué
puedo probar.
```

```
4. Crea el repositorio en GitHub, conéctalo a Vercel y despliega. Dame la lista exacta de variables de entorno que debo cargar en Vercel y los pasos en Google Cloud y Supabase que debo hacer yo.
5. Escribe un README con instrucciones de instalación local y despliegue.
</flujo_de_trabajo>
```

Prompt 2: Asesor financiero personal

Controla tus gastos, te dice en qué te pasas, cuánto te falta para tu meta de ahorro y te avisa antes de cada pago.

asesor_financiero.xml

```
<objetivo>
Construye y despliega en Vercel una aplicación web personal llamada "Asesor Financiero": registra mis gastos y cuentas, me dice en qué categorías me estoy pasando del presupuesto, cuánto me falta para mis metas de ahorro y me avisa por correo antes de que llegue un pago importante.
</objetivo>

<stack>
- Next.js (App Router) con TypeScript
- Tailwind CSS, shadcn/ui y Recharts para gráficas
- Supabase: Postgres, Auth, Storage y Row Level Security
- Anthropic API para leer estados de cuenta en PDF y categorizar movimientos (modelo en ANTHROPIC_MODEL, por defecto claude-sonnet-5-5)
- Resend para alertas por correo
- Vercel Cron Jobs para revisiones automáticas
</stack>

<usuario>
- Uso personal, un solo usuario: [TU CORREO]
- Moneda: [EJ. MXN]
- Zona horaria: [EJ. America/Mexico_City]
</usuario>

<modelo_de_datos>
Crea las migraciones de Supabase con RLS por user_id para:
- accounts: id, nombre, tipo (debito/credito/efectivo/ahorro), dia_corte, dia_limite_pago, saldo_actual
- categories: id, nombre, presupuesto_mensual, color_neutro
- transactions: id, account_id, fecha, descripcion_original, comercio, monto, tipo (gasto/ingreso), category_id, requiere_revision (boolean), es_recurrente (boolean),
```

```
hash_unico (para detectar duplicados)
- categorization_rules: id, patron_texto, category_id (reglas aprendidas cuando
corrijo una categoría)
- savings_goals: id, nombre, monto_objetivo, monto_actual, fecha_objetivo
- scheduled_payments: id, nombre, monto, frecuencia (mensual/anual/unica),
proximo_vencimiento, account_id
- imports: id, nombre_archivo, ruta_storage, estado, movimientos_detectados,
created_at
</modelo_de_datos>

<funcion_1_importacion_de_movimientos>
Pantalla "Importar" con tres formas de agregar movimientos:
1. Subir CSV o Excel: detecta las columnas automáticamente (fecha, descripción,
monto) y muéstrame una vista previa para confirmar el mapeo antes de guardar.
2. Subir estado de cuenta en PDF: guárdalo en Supabase Storage, envíalo a la API de
Anthropic y extrae los movimientos en JSON estructurado. Muestra la vista previa
para confirmar.
3. Captura rápida en una línea (ej. "350 súper con débito"), interpretada por la
API de Anthropic.
En todos los casos: calcula hash_unico con fecha, monto y descripción para
descartar duplicados, y marca como recurrentes los cargos que se repiten con monto
similar cada mes.
</funcion_1_importacion_de_movimientos>

<funcion_2_categorizacion>
1. Primero aplica las reglas de categorization_rules.
2. Lo que no tenga regla, envíalo en lote a la API de Anthropic con la lista de mis
categorías y pide salida JSON con categoría y nivel de confianza.
3. Si la confianza es baja, marca requiere_revision = true.
4. Bandeja "Por revisar" donde corrijo categorías con un clic; cada corrección crea
o actualiza una regla para que la próxima vez se categorice sola.
</funcion_2_categorizacion>

<funcion_3_control_de_presupuesto>
Dashboard principal con, para cada categoría del mes actual:
- Gastado, presupuesto, diferencia en monto y porcentaje
- Ritmo de gasto: porcentaje gastado contra porcentaje del mes transcurrido. Si va
más de 20 puntos por encima, estado "en riesgo"; si ya superó el presupuesto,
estado "excedido"
- Proyección al cierre del mes al ritmo actual
- Los 3 movimientos que más empujan el gasto de esa categoría
```

```
Ordena de mayor a menor exceso. Incluye una gráfica de barras de presupuesto contra gastado y una gráfica de evolución de los últimos 6 meses.
</funcion_3_control_de_presupuesto>

<funcion_4_metas_de_ahorro>
Para cada meta muestra: monto actual, objetivo, lo que falta, porcentaje de avance, cuánto debo ahorrar por mes y por quincena para llegar a la fecha, y si al ritmo de los últimos 3 meses llego a tiempo o con cuántos meses de retraso. Botón "¿De dónde saco la diferencia?" que le pide a la API de Anthropic una propuesta basada en mis categorías más excedidas, con montos concretos.
</funcion_4_metas_de_ahorro>

<funcion_5_alertas_de_pagos>
1. Calendario mensual con fechas de corte, fechas límite de tarjetas y pagos programados.
2. Ruta /api/cron/payment-alerts protegida con CRON_SECRET, ejecutada todos los días a las 8:00 am de mi zona horaria (convertida a UTC en vercel.json).
3. Envía un correo con Resend cuando falten 7 días y 2 días para un pago, con el monto y si el saldo de la cuenta asociada alcanza.
4. Ruta /api/cron/weekly-report ejecutada cada [DÍA Y HORA]: correo con estado del presupuesto, avance de metas, pagos de los próximos 14 días y una acción recomendada para la semana.
</funcion_5_alertas_de_pagos>

<diseno>
- Paleta neutra: negro, grises y blanco. Usa un solo tono de rojo solo para "excedido".
- Montos con separador de miles y moneda.
- Responsive, pensado para registrar gastos desde el celular.
</diseno>

<seguridad>
- Nunca pidas ni guardes números completos de tarjeta, contraseñas o claves bancarias; si aparecen en un PDF, guarda solo los últimos 4 dígitos.
- Los PDF en un bucket privado de Supabase Storage con políticas por usuario.
- La app no da recomendaciones de inversión ni de compra o venta de instrumentos financieros; agrega esa instrucción al system prompt de cada llamada a la API.
- Llaves solo en variables de entorno.
</seguridad>

<flujo_de_trabajo>
```

```
1. Antes de escribir código, muéstrame el plan: estructura, migraciones, rutas y variables de entorno. Espera mi aprobación.
2. Construye por fases: (a) base y login, (b) cuentas, categorías y captura manual, (c) importación CSV y PDF, (d) categorización y reglas, (e) dashboard y metas, (f) crons y correos.
3. Incluye datos de ejemplo (seed) para probar el dashboard sin mis datos reales.
4. Al terminar cada fase, corre build y tipos, corrige errores y dime qué probar.
5. Crea el repo en GitHub, despliega en Vercel y dame la lista exacta de variables de entorno y pasos manuales en Supabase y Resend.
6. Escribe un README con instalación local y despliegue.
</flujo_de_trabajo>
```

Prompt 3: Entrenador de gym y alimentación

Te arma rutinas y menús según tus objetivos y los ajusta con tu progreso real.

entrenador_gym.xml

```
<objetivo>
Construye y despliega en Vercel una aplicación web personal llamada "Coach": recibe mis objetivos, lo que como y cómo entreno, me arma rutinas y menús ajustados a mi progreso real registrado semana a semana, y ajusta el plan con base en mis datos, no en una plantilla genérica.
</objetivo>

<stack>
- Next.js (App Router) con TypeScript
- Tailwind CSS, shadcn/ui y Recharts
- Supabase: Postgres, Auth y Row Level Security
- Anthropic API para generar rutinas, menús y explicar ajustes (modelo en ANTHROPIC_MODEL, por defecto claude-sonnet-5-5), siempre con salida JSON estructurada validada con Zod
- Resend para el recordatorio semanal
- Vercel Cron Jobs
</stack>

<usuario>
- Uso personal, un solo usuario: [TU CORREO]
- Zona horaria: [EJ. America/Mexico_City]
- Día del registro semanal: [EJ. domingo]
</usuario>

<modelo_de_datos>
```

```
Migraciones de Supabase con RLS por user_id para:
- profile: edad, sexo, estatura_cm, peso_inicial_kg, objetivo
(masa/grasa/resistencia), meta_medible, fecha_meta, experiencia, dias_por_semana,
minutos_por_sesion, lugar_entrenamiento, equipo, lesiones, alimentos_habituales,
alimentos_excluidos, veces_que_cocina_por_semana
- nutrition_targets: calorias, proteina_g, grasa_g, carbohidratos_g, formula_usada,
vigente_desde
- workout_plans: id, version, division, json_plan, motivo_del_cambio, vigente_desde
- workout_logs: id, fecha, ejercicio, serie, peso_kg, repeticiones, rir
- meal_plans: id, version, json_menu, lista_compras, vigente_desde
- weekly_checkins: id, semana, peso_promedio, cintura_cm, cadera_cm, brazo_cm,
adherencia_dieta_pct, energia, sueno, hambre, notas
- adjustments: id, fecha, tipo (calorias/volumen/ejercicio), detalle,
dato_que_lo_motivo
</modelo_de_datos>

<funcion_1_onboarding_y_calculo>
Formulario por pasos para llenar profile. Al terminar, calcula en código (no con la
IA):
- Gasto energético con Mifflin-St Jeor por factor de actividad según días de
entrenamiento
- Calorías objetivo: déficit de 15 a 20% para bajar grasa, superávit de 5 a 10%
para masa, mantenimiento para resistencia
- Proteína de 1.8 a 2.2 g por kg, grasa de 0.8 a 1 g por kg, carbohidratos con el
resto
Muestra los números con la fórmula y una explicación de una línea para cada uno.
</funcion_1_onboarding_y_calculo>

<funcion_2_rutina>
Genera la rutina con la API de Anthropic enviando el perfil completo. La respuesta
debe incluir:
- División elegida y motivo
- Por día: ejercicio, series, repeticiones, descanso, RIR objetivo y alternativa si
el equipo está ocupado
- Sustitución de ejercicios que afecten mis lesiones
- Calentamiento de 5 a 10 minutos
- Regla de progresión: cuándo subir peso, repeticiones o hacer descarga
Pantalla "Hoy" que muestra la sesión del día con campos para registrar peso,
repeticiones y RIR por serie en workout_logs, mostrando lo que hice la última vez
en ese ejercicio.
</funcion_2_rutina>
```

```
<funcion_3_menu>
Genera el menú semanal con la API de Anthropic usando principalmente mis alimentos habituales y excluyendo los que no como:
- Comidas por día con porciones en gramos y macros por comida
- Total diario dentro de ±5% de las calorías objetivo; valida la suma en código y regenera si no cumple
- Reemplazos con macros similares para cada comida
- Lista de compras agrupada por sección del súper
- Si cocino 3 veces o menos por semana, organiza el menú en preparaciones por lote
</funcion_3_menu>

<funcion_4_registro_y_ajuste_por_progreso>
Formulario semanal para weekly_checkins. Al guardarlo, ejecuta estas reglas en código:
- Compara contra la semana anterior y contra la tendencia de 4 semanas
- Si el peso no se mueve en la dirección esperada durante 2 semanas seguidas con adherencia de 80% o más, ajusta calorías entre 100 y 200
- Si un ejercicio principal no mejora en peso ni repeticiones durante 3 sesiones, cambia el esquema o el ejercicio
- Si energía o sueño promedian menos de 5, reduce el volumen semanal 20%
Registra cada ajuste en adjustments con el dato que lo motivó y pide a la API de Anthropic que regenere solo la parte del plan que cambió y redacte la explicación.
Pantalla "Progreso" con gráficas de peso promedio, medidas y carga de los ejercicios principales, más el historial de ajustes.
</funcion_4_registro_y_ajuste_por_progreso>

<recordatorio_semanal>
Ruta /api/cron/weekly-checkin protegida con CRON_SECRET que el día del registro me envía un correo con el enlace al formulario y el resumen de la semana: sesiones completadas, cumplimiento del plan y siguiente objetivo.
</recordatorio_semanal>

<diseno>
- Paleta neutra: negro, grises y blanco.
- Pensado primero para celular: botones grandes para registrar series en el gimnasio.
</diseno>

<seguridad_y_salud>
- El system prompt de cada llamada debe prohibir déficits extremos, ayunos
```

```
prolongados y suplementos de riesgo.
- Si en el registro escribo dolor agudo, mareo o una condición médica, muestra un aviso recomendando consultar a un médico y pausa los aumentos de carga.
- Llaves solo en variables de entorno.
</seguridad_y_salud>

<flujo_de_trabajo>
1. Antes de escribir código, muéstrame el plan: estructura, migraciones, rutas, esquemas Zod y variables de entorno. Espera mi aprobación.
2. Construye por fases: (a) base y login, (b) onboarding y cálculo, (c) rutina y registro de series, (d) menú y lista de compras, (e) registro semanal, ajustes y gráficas, (f) cron y correo.
3. Escribe pruebas unitarias para las fórmulas y las reglas de ajuste.
4. Al terminar cada fase, corre build, tipos y pruebas, corrige errores y dime qué probar.
5. Crea el repo en GitHub, despliega en Vercel y dame la lista exacta de variables de entorno y pasos manuales.
6. Escribe un README con instalación local y despliegue.
</flujo_de_trabajo>
```

Prompt 4: Segundo cerebro

Guarda todo tu papeleo y responde al momento cualquier pregunta, como cuándo vence tu seguro.

segundo_cerebro.xml

```
<objetivo>
Construye y despliega en Vercel una aplicación web personal llamada "Segundo Cerebro": subo ahí todo mi papeleo (contratos, seguros, notas, PDF), la app extrae los datos clave de cada documento y puedo preguntarle cualquier cosa, como "¿cuándo vence mi seguro?", y me responde al momento citando el documento de origen.
</objetivo>

<stack>
- Next.js (App Router) con TypeScript
- Tailwind CSS y shadcn/ui
- Supabase: Postgres con búsqueda de texto completo (tsvector), Auth, Storage y Row Level Security
- Anthropic API para extraer datos de documentos y responder preguntas (modelo en ANTHROPIC_MODEL, por defecto claude-sonnet-5-5)
- Resend para alertas de vencimiento
- Vercel Cron Jobs
```

```
</stack>

<usuario>
- Uso personal, un solo usuario: [TU CORREO]
- Zona horaria: [EJ. America/Mexico_City]
</usuario>

<modelo_de_datos>
Migraciones de Supabase con RLS por user_id para:
- documents: id, nombre, ruta_storage, tipo_archivo, categoria, tipo_documento,
texto_extraido, busqueda (tsvector generado en español sobre nombre y texto),
estado_procesamiento, created_at
- document_chunks: id, document_id, pagina, contenido, busqueda (tsvector en
español)
- document_facts: id, document_id, partes, numero_referencia (solo últimos 4
dígitos si es sensible), fecha_inicio, fecha_vencimiento, montos (jsonb),
condiciones_clave (jsonb), contacto, campos_no_especificados (jsonb)
- notes: id, titulo, contenido, categoria, busqueda (tsvector)
- questions: id, pregunta, respuesta, documentos_citados (jsonb), created_at
Categorías fijas: Seguros, Contratos, Identidad y trámites, Salud, Vehículos,
Hogar, Finanzas e impuestos, Notas personales.
</modelo_de_datos>

<funcion_1_carga_y_procesamiento>
1. Zona de arrastrar y soltar que acepta PDF, imágenes y archivos de texto, varios
a la vez.
2. Guarda cada archivo en un bucket privado de Supabase Storage.
3. Envía el archivo a la API de Anthropic y obtén en JSON estructurado: categoría,
tipo de documento, texto por página y la ficha de datos (partes, número de
referencia, fecha de inicio, fecha de vencimiento, montos como prima, renta,
deducible o suma asegurada, hasta 3 condiciones clave como plazo de aviso o
exclusiones, y contacto).
4. Si un campo no aparece, guárdalo como "no especificado". Nunca completes campos
con suposiciones; agrega esa regla al system prompt.
5. Divide el texto por página en document_chunks.
6. Muestra el estado de procesamiento en tiempo real y permite editar la ficha
manualmente.
</funcion_1_carga_y_procesamiento>

<funcion_2_biblioteca_e_indice>
- Vista de biblioteca filtrable por categoría con búsqueda.
```

- Índice maestro en tabla con todas las fichas, ordenado por fecha de vencimiento más cercana. Resalta lo que vence en menos de 30 días y lo ya vencido.
- Vista de detalle de cada documento: ficha a la izquierda, visor del PDF a la derecha.
</funcion_2_biblioteca_e_indice>

<funcion_3_preguntas_al_momento>
Chat en la página principal con este flujo:
1. Busca en document_facts, document_chunks y notes con búsqueda de texto completo en español y toma los 8 fragmentos más relevantes.
2. Envía la pregunta, los fragmentos y las fichas relacionadas a la API de Anthropic con instrucciones de responder solo con esa información.
3. Formato de respuesta: el dato exacto en la primera línea, después el documento y la página de origen como enlace que abre el documento en esa página.
4. Si dos documentos podrían responder (ej. dos pólizas de auto), muestra ambos y pregunta cuál.
5. Si el dato está vencido o vence en menos de 30 días, agrégalo como aviso.
6. Si no hay información, responde que no está en los documentos y sugiere qué documento subir.
7. Transmite la respuesta en streaming y guarda el historial en questions.
</funcion_3_preguntas_al_momento>

<funcion_4_alertas_de_vencimiento>
Ruta /api/cron/expirations protegida con CRON_SECRET, ejecutada diario a las 8:00 am de mi zona horaria (en UTC en vercel.json). Envía un correo con Resend cuando un documento esté a 30 días y a 7 días de vencer, con la acción sugerida (renovar, cancelar, avisar). El día 1 de cada mes envía la lista de todo lo que vence en los próximos 60 días.
</funcion_4_alertas_de_vencimiento>

<funcion_5_documentos_faltantes>
Botón "¿Qué me falta?" que revisa las categorías y fichas existentes y sugiere documentos que suelen ir juntos y no están (ej. hay póliza de auto pero no tarjeta de circulación).
</funcion_5_documentos_faltantes>

<diseno>
- Paleta neutra: negro, grises y blanco.
- Responsive, con el chat accesible en una tecla desde cualquier pantalla en escritorio.
</diseno>

```
<seguridad>
- Bucket privado con políticas por usuario; los archivos se sirven solo con URLs firmadas de corta duración.
- Nunca muestres números completos de identificación, cuentas o tarjetas; solo los últimos 4 dígitos.
- La app explica lo que dice cada cláusula pero no da interpretaciones legales; incluye esa regla en el system prompt.
- Llaves solo en variables de entorno.
</seguridad>

<flujo_de_trabajo>
1. Antes de escribir código, muéstrame el plan: estructura, migraciones, configuración de búsqueda, rutas y variables de entorno. Espera mi aprobación.
2. Construye por fases: (a) base y login, (b) carga a Storage, (c) extracción y fichas, (d) biblioteca e índice, (e) chat con citas, (f) cron, alertas y documentos faltantes.
3. Prueba con 3 documentos de ejemplo que generes tú (una póliza, un contrato de renta y una garantía) antes de usar los míos.
4. Al terminar cada fase, corre build y tipos, corrige errores y dime qué probar.
5. Crea el repo en GitHub, despliega en Vercel y dame la lista exacta de variables de entorno y pasos manuales en Supabase y Resend.
6. Escribe un README con instalación local y despliegue.
</flujo_de_trabajo>
```